



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Tema 4: Aprendizaje por refuerzo sin modelo



Hasta ahora...

- Sabemos lo que son la **recompensa** y el **retorno**
- También conocemos algunas **reglas de actualización** para estimarlos a partir de la experiencia.
- Sabemos que hay un compromiso (dilema) entre **exploración y explotación**
- Conocemos estrategias para **equilibrar exploración y explotación** (ϵ -greedy).

Hasta ahora...

- Sabemos cómo modelar problemas de toma de decisiones (**MDPs**):
 - Acciones, estados y conjuntos de recompensas
 - Transiciones (dinámica): una distribución de probabilidad por cada par estado-acción.
 - PROPIEDAD DE MARKOV
- Conocemos la definición de dos funciones especiales:
 - La función de **valor de estado**.
 - La función de **valor de acción**.



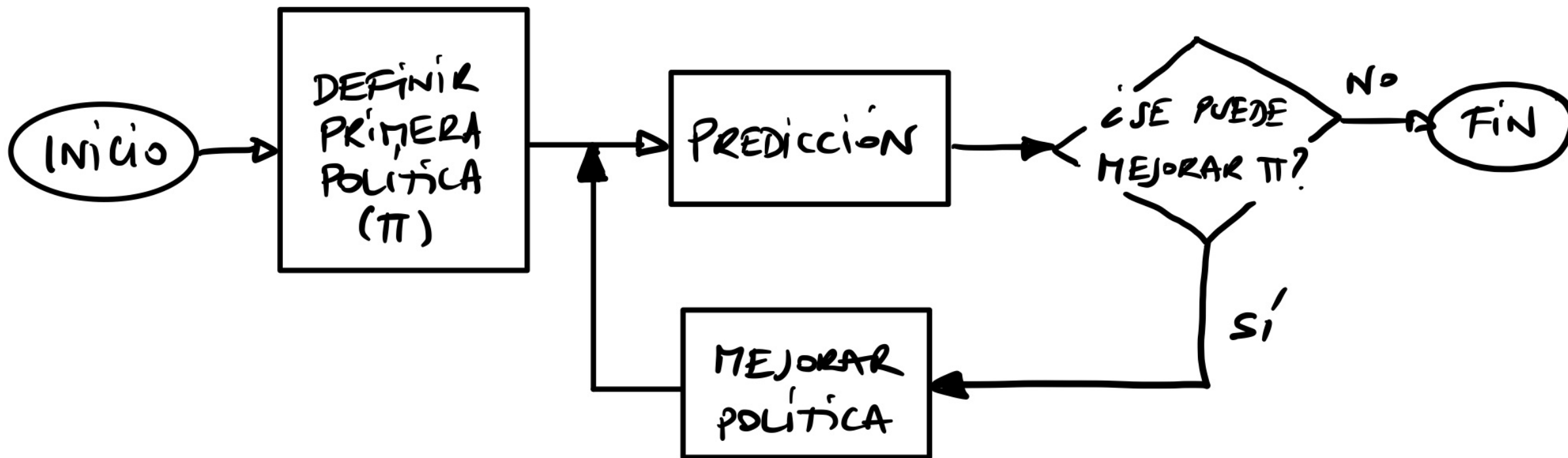
Hasta ahora...

- Conocemos dos tipos especiales de ecuaciones recursivas llamadas **ecuaciones de Bellman** que, junto con el modelo de dinámica, nos permiten calcular las funciones de valor de estado y acción para una política dada.
- También conocemos la **versión óptima de las ecuaciones de Bellman**, que nos permiten obtener **políticas óptimas**.
- Con la **función de valor de acción óptima**, podemos obtener directamente una **política determinista óptima**, mientras que con la función de valor de estado podemos hacerlo si tenemos el modelo de dinámica del MDP.

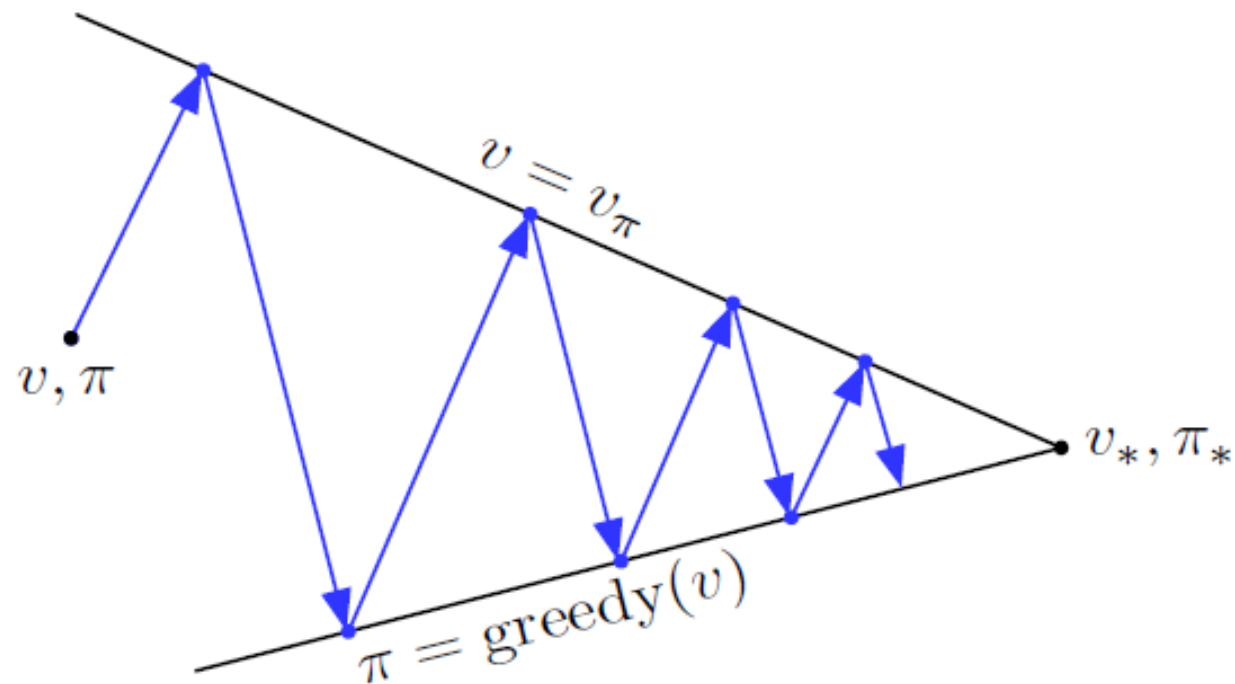
Hasta ahora...

- Tenemos métodos directo e **ITERATIVO** para resolver el problema de predicción:
Evaluación de Política Iterativa
- Conocemos un **operador que nos permite mejorar una política** una vez que conocemos su función de valor asociada, y sabemos que si la política no cambia al aplicar este operador, la razón es que es óptima.
- Conocemos un algoritmo llamado **Iteración de Política** que, combinando un algoritmo de evaluación y este operador de mejora, hace posible **resolver el problema de control (planificación)** es decir, obtener una política óptima.

Hasta ahora...



Hasta ahora...



Entonces, lo que necesitamos para el control

- En resumen, “todo” lo que necesitamos para **resolver el problema de control** es:
 - Un método para **estimar los valores de acción** para una política dada.
 - Un método para **mejorar la política**
- Ten en cuenta que en la "vida real", **no tendremos el modelo** del entorno en el que se basan los métodos de DP para resolver el problema de predicción.
- Aquí es donde **los métodos de RL** proporcionan un conjunto de estrategias para estimar los valores de acción de a partir de **un flujo de experiencia**



COMILLAS

UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Predicción sin modelo

comillas.edu

Métodos de Monte Carlo (MC)

- Podemos obtener una **estimación de funciones de valor** a partir de la experiencia **promediando empíricamente** los retornos obtenidos cada vez que visitamos cada par estado-acción.

$$S_1, A_1, R_2, \dots, S_k \sim \pi \quad \longrightarrow \quad v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

- El principal problema/ineficiencia del enfoque es que **necesitamos un número muy grande de visitas a TODOS** los pares estado-acción, y no todas las políticas garantizan eso.

Predicción de Monte Carlo

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

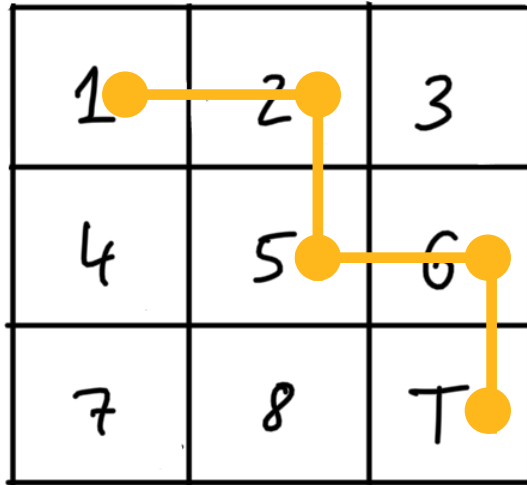
Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Predicción de Monte Carlo

- Cómo evolucionan los valores de estado de nuestro *gridworld* de juguete:

$S_0=1, A_0=r, R_1=-1, S_1=2, A_1=d, R_2=-1, S_2=5, A_2=r, R_3=-1, S_3=6, A_3=r, R_4=-1, S_4=T$



Predicción de Monte Carlo

limitaciones

- Sólo para MDPs **episódicos**
- Estimadores no sesgados, pero con **alta varianza** → **aprendizaje lento**
- Dependiendo del tipo de política (especialmente con las deterministas), **hay situaciones que nunca vamos a explorar.**
- Sin visitas, no tenemos **nada para promediar**
- Esto es especialmente importante cuando estimamos los valores de acción, donde necesitamos **visitor** un gran número de veces (infinito en teoría) **cada par estado-acción**

Políticas ϵ -soft

- Para resolver el problema de no visitar cada par estado-acción, necesitaremos una política de la familia **ϵ -soft**.
- Las políticas ϵ -soft son una familia de políticas donde **todas las acciones tienen asignada una masa de probabilidad no nula**. Por supuesto, ϵ -greedy es un ejemplo
- El teorema de mejora de la política **se puede extender** a estas políticas → garantías de convergencia a una política óptima con métodos MC.

Predicción con métodos TD

- Los métodos de Monte Carlo necesitan esperar hasta el final del episodio para actualizar las estimaciones de los valores de las acciones... pero recordemos:

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] \quad (6.3)$$

$$= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \quad (\text{from (3.9)})$$

$$= \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s]. \quad (6.4)$$

- El retorno se puede calcular a partir de la recompensa inmediata y el valor del próximo estado, así que...
- ¿Y si usamos nuestra estimación de este estado para actualizar nuestra estimación del estado actual? ¿Y si hacemos **BOOTSTRAP!**?

Predicción TD y MC

Regla de actualización

- Cómo actualizamos la predicción con métodos de **Monte Carlo**:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)],$$

- Ahora, siguiendo la idea de bootstrapping, una regla simple de actualización de **TD** sería:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Ambas reglas siguen el **patrón de actualización de RL** presentado en el tema 1:

$$NewEstimate \leftarrow OldEstimate + StepSize [Target - OldEstimate]$$

Algoritmo de predicción TD(0)

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

 until S is terminal

El error TD

- La expresión recuadrada:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]$$

puede interpretarse como **el error entre lo que esperaba obtener** ($V(S_t)$) **y mi estimación actualizada** con mi nueva experiencia + bootstrap ($R_{t+1} + \gamma V(S_{t+1})$):

$$\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

- Esto es el error TD, una variable muy importante cuyo **comportamiento puede afectar fuertemente el aprendizaje**.

Un ejemplo

Example 6.1: Driving Home Each day as you drive home from work, you try to predict how long it will take to get home. When you leave your office, you note the time, the day of week, the weather, and anything else that might be relevant. Say on this Friday you are leaving at exactly 6 o'clock, and you estimate that it will take 30 minutes to get home. As you reach your car it is 6:05, and you notice it is starting to rain. Traffic is often slower in the rain, so you reestimate that it will take 35 minutes from then, or a total of 40 minutes. Fifteen minutes later you have completed the highway portion of your journey in good time. As you exit onto a secondary road you cut your estimate of total travel time to 35 minutes. Unfortunately, at this point you get stuck behind a slow truck, and the road is too narrow to pass. You end up having to follow the truck until you turn onto the side street where you live at 6:40. Three minutes later you are home. The sequence of states, times, and predictions is thus as follows:

<i>State</i>	<i>Elapsed Time (minutes)</i>	<i>Predicted Time to Go</i>	<i>Predicted Total Time</i>
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

Un ejemplo

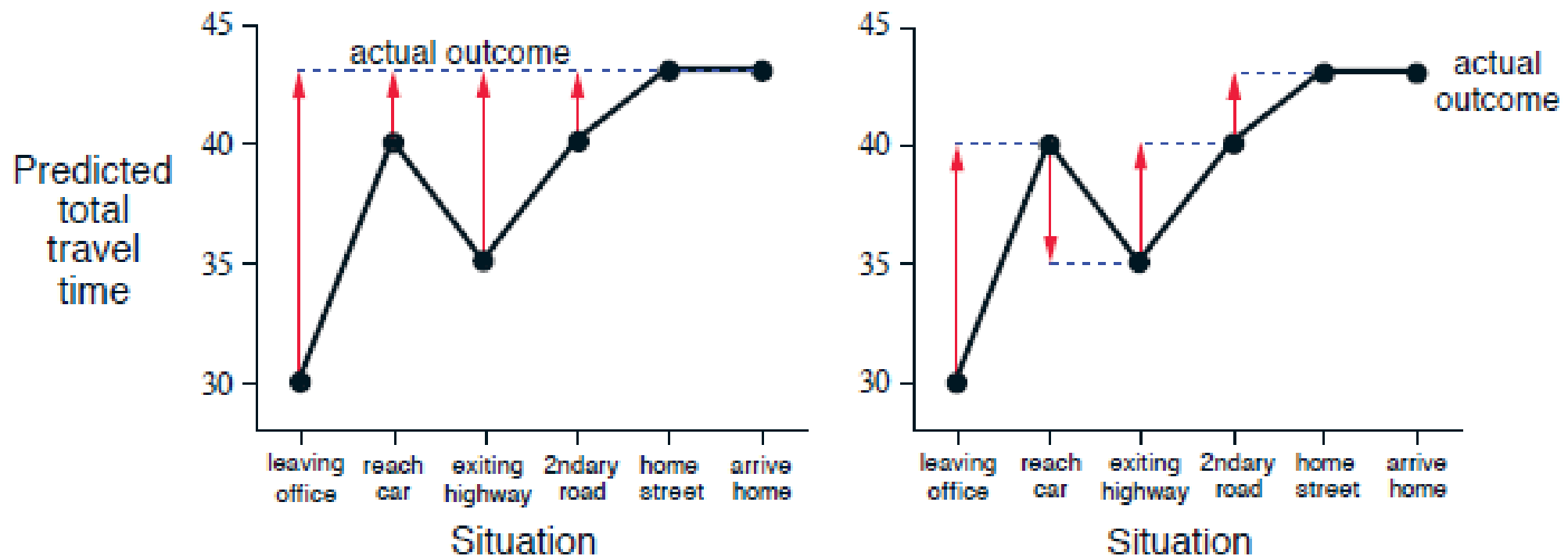


Figure 6.1: Changes recommended in the driving home example by Monte Carlo methods (left) and TD methods (right).

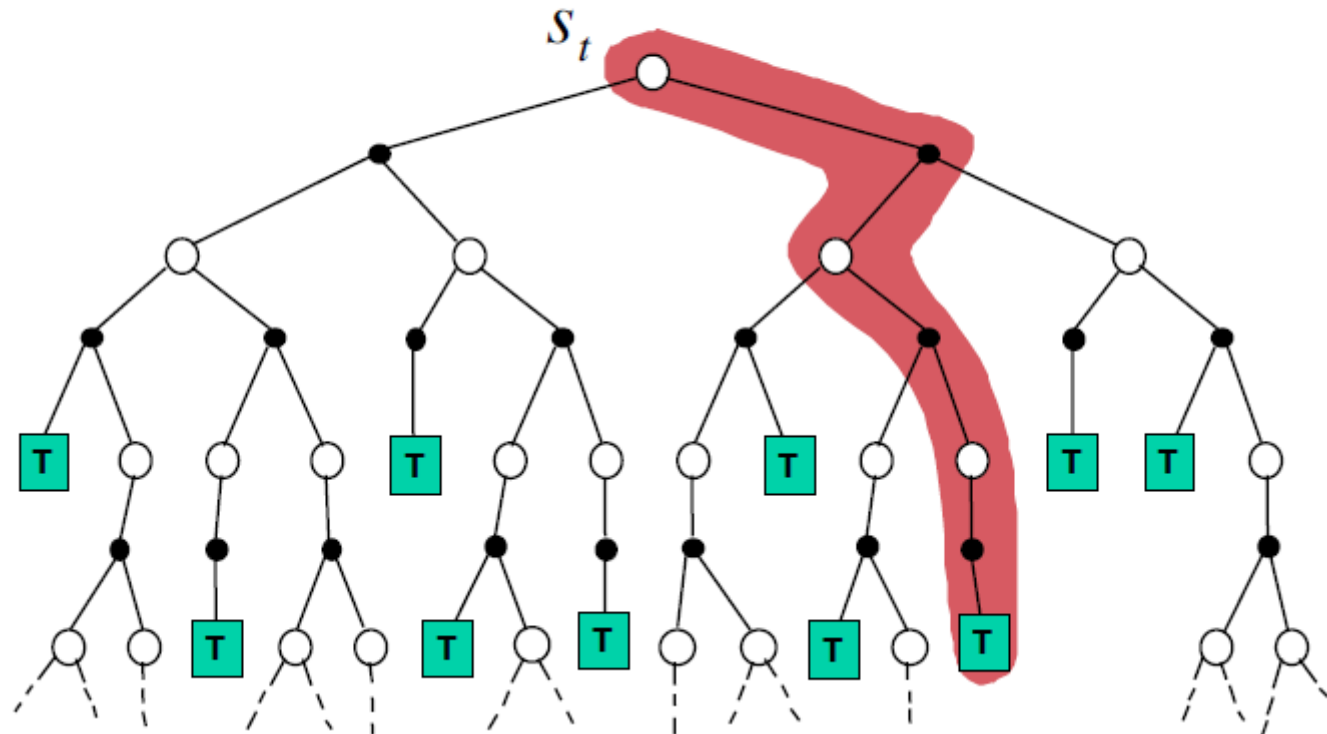
cor

TD vs MC

- MC:
 - Converge con más garantías
 - No sensible a propiedad MDP
 - No tiene sesgo, pero varianza enorme
 - Requiere que el episodio termine
- TD:
 - Converge, pero a veces problemas cuando aproximemos
 - Sensible a propiedad MDP
 - Varianza reducida (aprendizaje más rápido), pero sesgo
 - Vale para contextos continuos

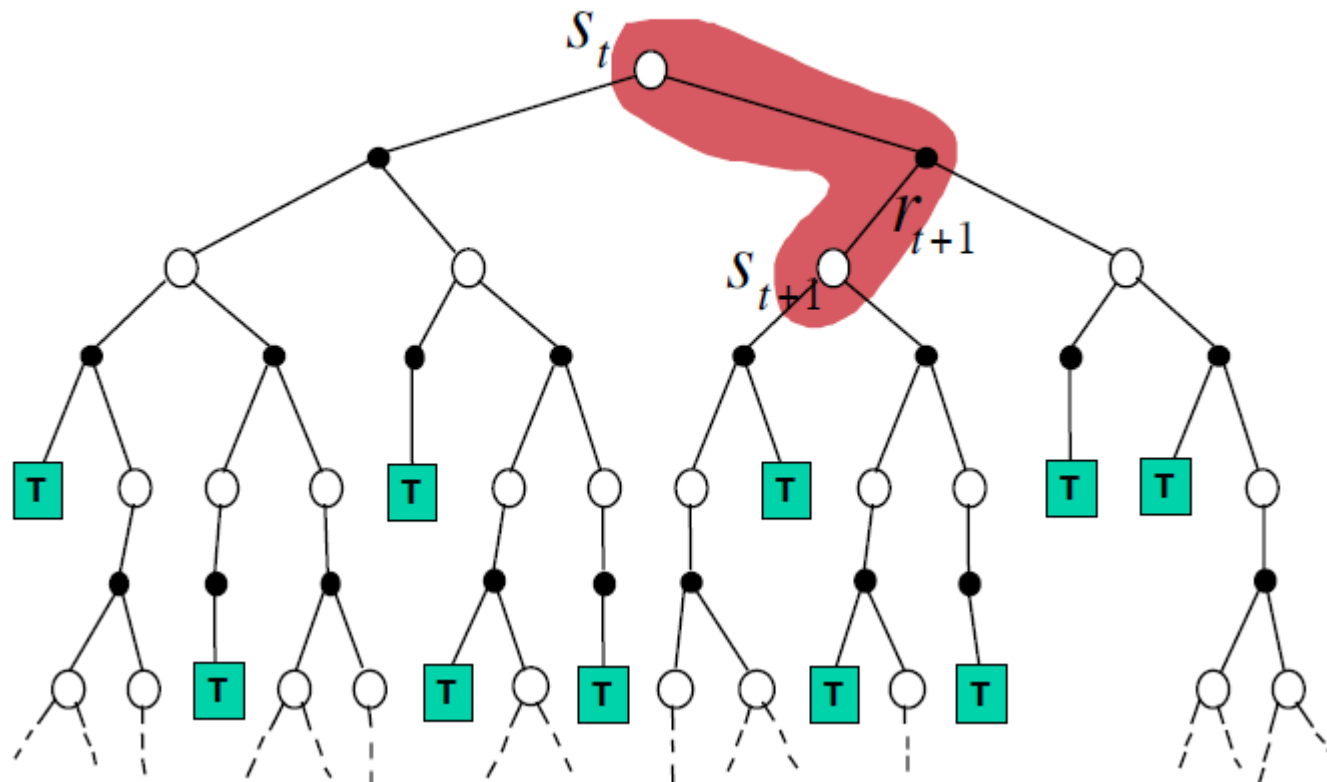
Comparación entre métodos MC

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$



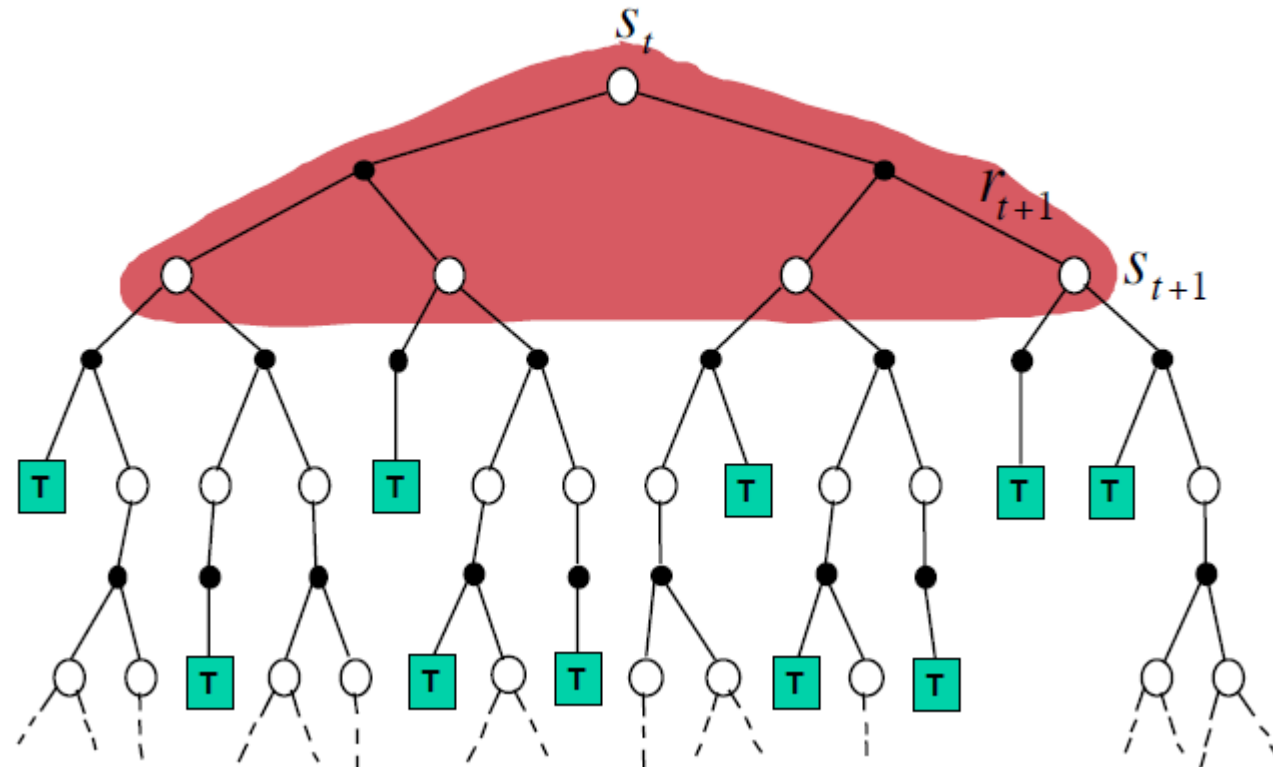
Comparación entre métodos TD

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

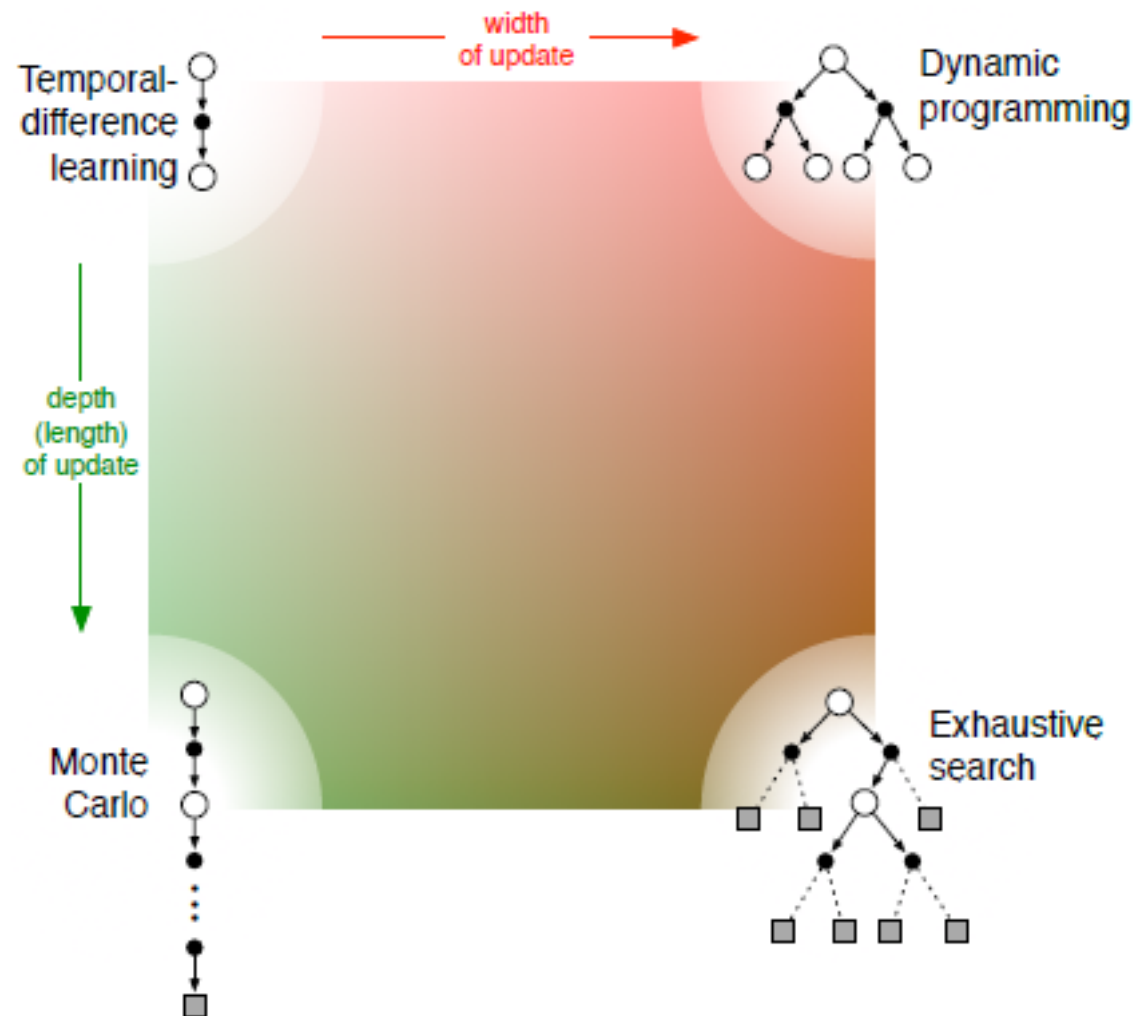


Comparación entre métodos DP

$$V(S_t) \leftarrow \mathbb{E}_{\pi} [R_{t+1} + \gamma V(S_{t+1})]$$



Visión unificada





COMILLAS

UNIVERSIDAD PONTIFICIA

ICAI

ICADE

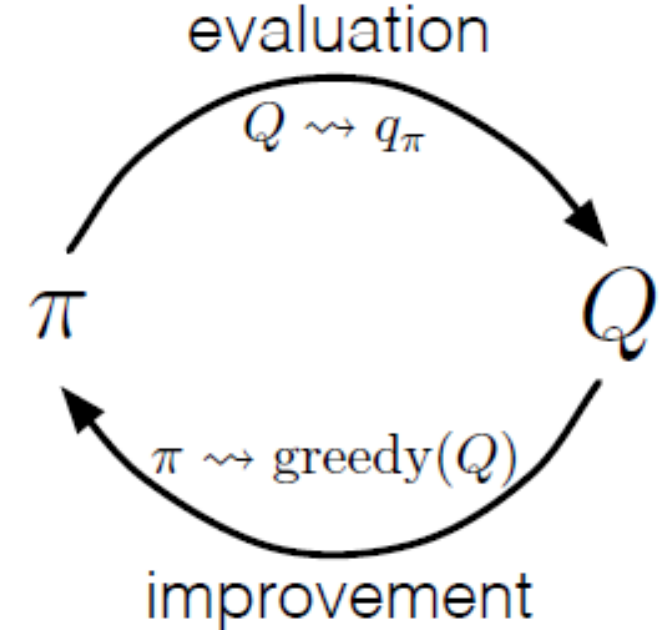
CIHS

Control sin modelo

comillas.edu

Control de Monte Carlo

- Podemos aplicar **Iteración de Política Generalizada** para encontrar políticas óptimas.
- En un enfoque de MC es muy natural aplicar la **mejora de la política después de un solo episodio**. Aunque, por supuesto, esto no es obligatorio.



Control de Monte Carlo

On-policy first-visit MC control (for ε -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε -soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

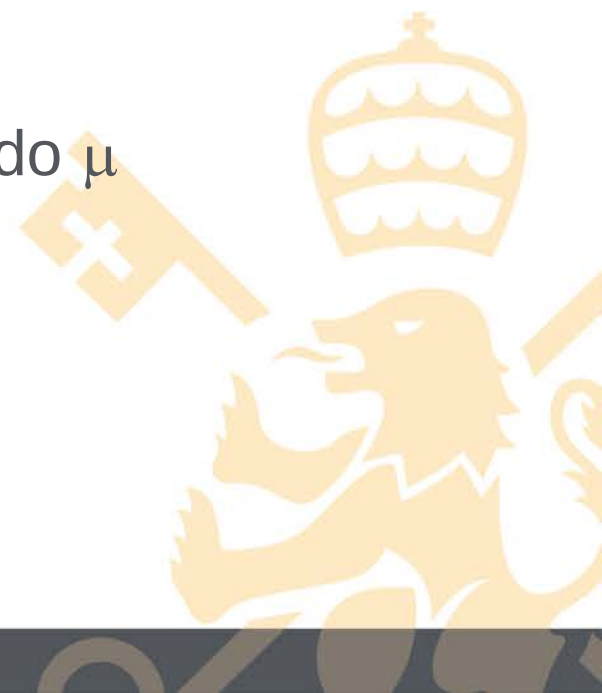
$A^* \leftarrow \operatorname{argmax}_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Aprendizaje por refuerzo on- y off-policy

- **On-policy:**
 - Optimizo una política π a partir de experiencia obtenida siguiendo π
- **Off-policy:**
 - Optimizo una política π a partir de experiencia obtenida siguiendo μ



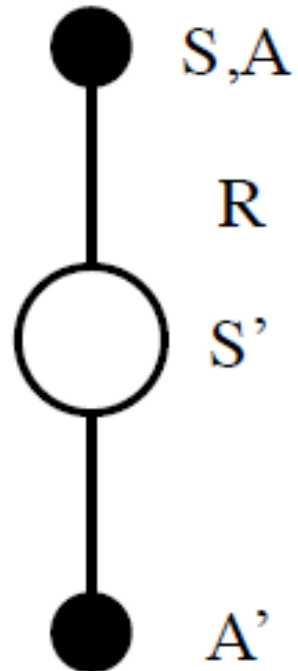
Control off-policy: Muestreo de importancia

- Ojo con el control off-policy:

$$q_{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right]$$

- Esto plantea un problema: podemos obtener fácilmente retornos promediados **siguiendo la política de comportamiento**, pero necesitamos calcular los valores de acción asociados **¡con la política objetivo!**
- Para resolver este problema, utilizamos **muestreo de importancia**, una técnica que permite pasar de una política a otra sin depender del modelo de dinámicas del MDP.

Control TD on-policy: Sarsa



$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

Control TD on-policy: Sarsa

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

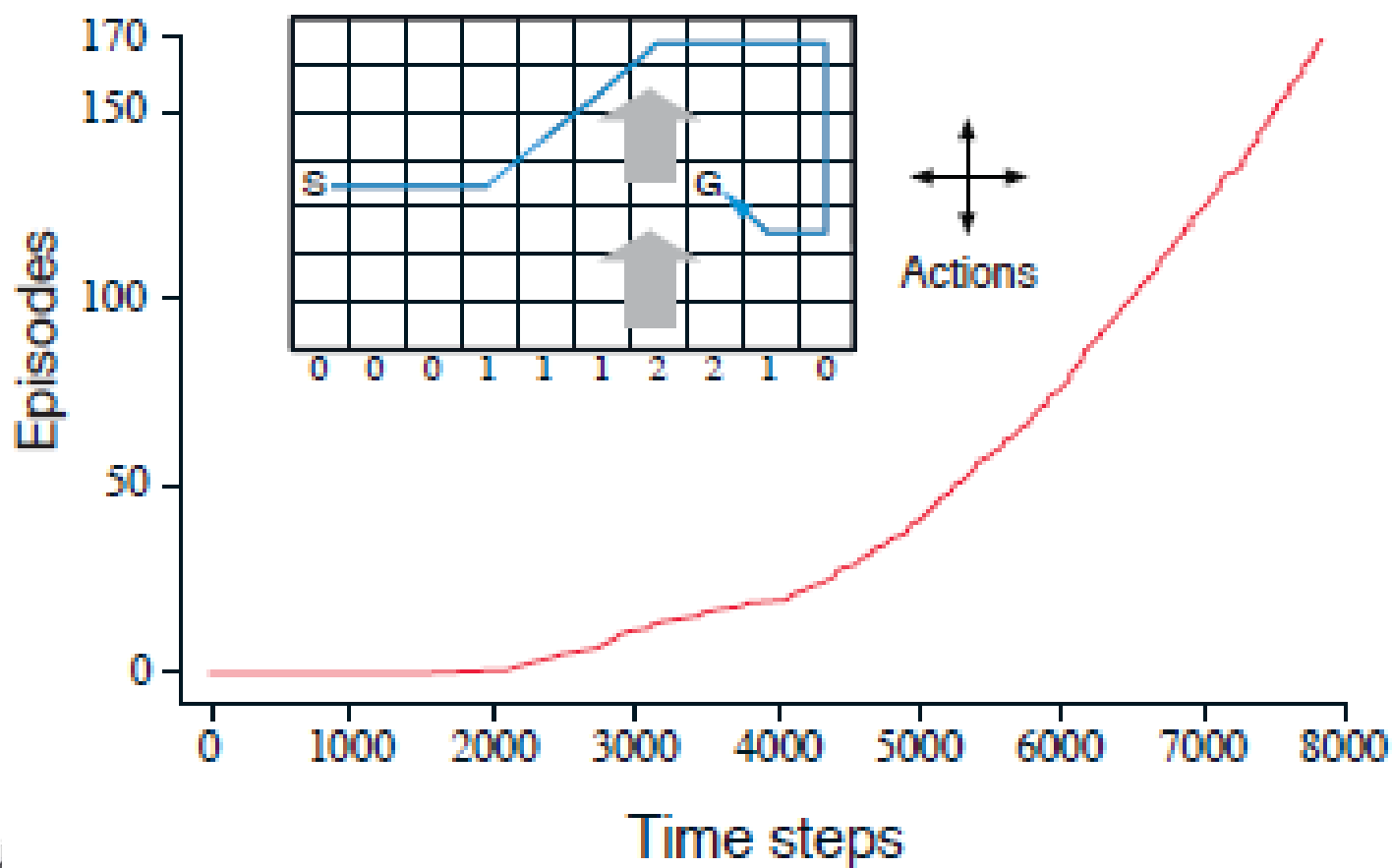
 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

Rendimiento de Sarsa *Windy Gridworld*



Control TD *off-policy*: Q-learning

- Política de comportamiento: ε -greedy con respecto a $Q(s,a)$
- Política objetivo: greedy con respecto a $Q(s,a)$
- El TD target queda:

$$\begin{aligned} & R_{t+1} + \gamma Q(S_{t+1}, A') \\ &= R_{t+1} + \gamma Q(S_{t+1}, \underset{a'}{\operatorname{argmax}} Q(S_{t+1}, a')) \\ &= R_{t+1} + \max_{a'} \gamma Q(S_{t+1}, a') \end{aligned}$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Control TD off-policy: Q-learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

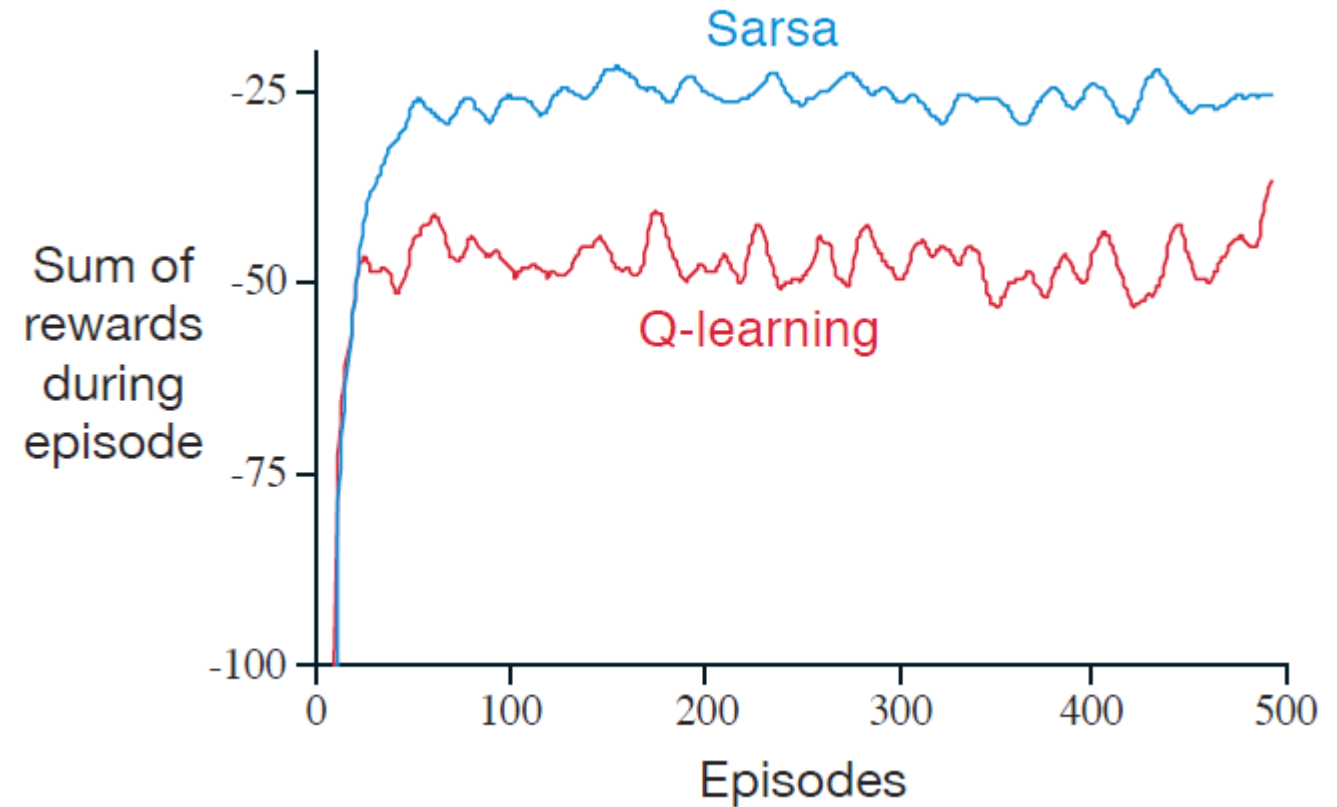
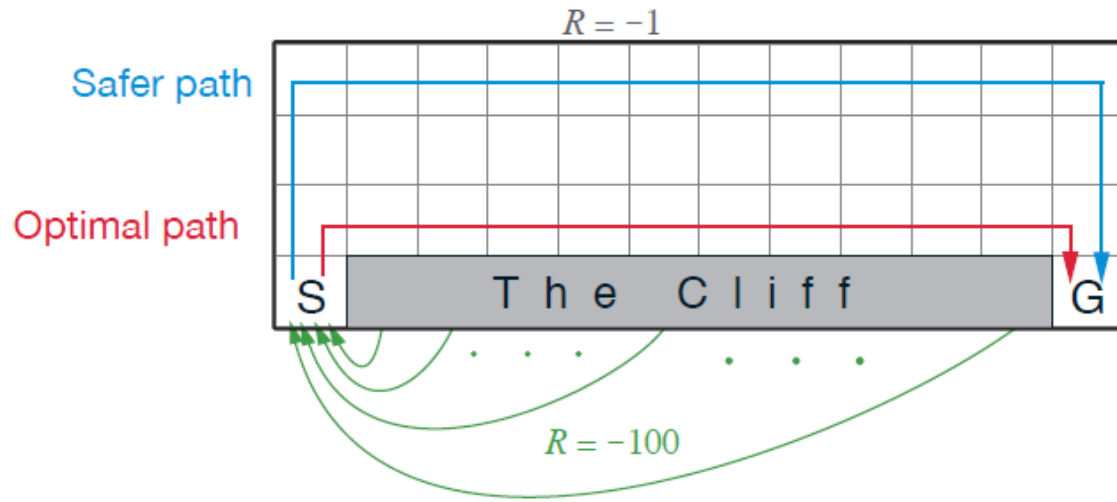
 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Comparación entre SARSA y Q-learning



Sesgo de maximización → Aprendizaje doble

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Loop for each step of episode:

Choose A from S using the policy ε -greedy in $Q_1 + Q_2$

Take action A , observe R, S'

With 0.5 probability:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha \left(R + \gamma Q_2(S', \arg\max_a Q_1(S', a)) - Q_1(S, A) \right)$$

else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha \left(R + \gamma Q_1(S', \arg\max_a Q_2(S', a)) - Q_2(S, A) \right)$$

$S \leftarrow S'$

until S is terminal

¡El aprendizaje doble funciona!

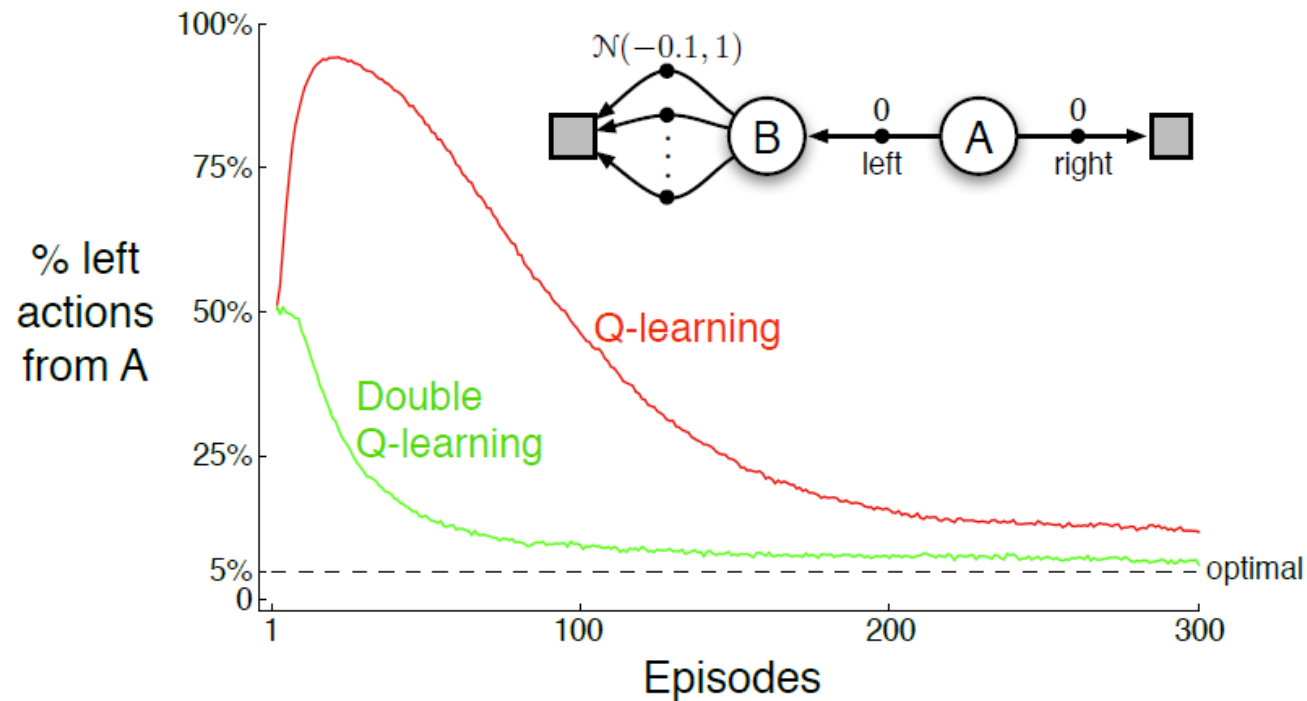


Figure 6.5: Comparison of Q-learning and Double Q-learning on a simple episodic MDP (shown inset). Q-learning initially learns to take the left action much more often than the right action, and always takes it significantly more often than the 5% minimum probability enforced by ϵ -greedy action selection with $\epsilon = 0.1$. In contrast, Double Q-learning is essentially unaffected by maximization bias. These data are averaged over 10,000 runs. The initial action-value estimates were zero. Any ties in ϵ -greedy action selection were broken randomly.